

Medical Insurance Charges Implementation

Collecting the data

Importing the pandas library

```
In [1]: import pandas as pd
```

Importing the dataset

```
In [2]: df = pd.read_csv("insurance.csv")
```

Understanding the data

Viewing the first few rows in the dataset

```
In [3]: df.head()
```

```
Out[3]:
```

	age	sex	bmi	children	smoker	region	charges
0	19	female	27.900	0	yes	southwest	16884.92400
1	18	male	33.770	1	no	southeast	1725.55230
2	28	male	33.000	3	no	southeast	4449.46200
3	33	male	22.705	0	no	northwest	21984.47061
4	32	male	28.880	0	no	northwest	3866.85520

Viewing the total number of rows and columns in the dataset

```
In [4]: df.shape
```

```
Out[4]: (1338, 7)
```

Inspecting the data

```
In [5]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1338 entries, 0 to 1337
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  -
0   age         1338 non-null   int64
1   sex         1338 non-null   object
2   bmi         1338 non-null   float64
3   children    1338 non-null   int64
4   smoker      1338 non-null   object
5   region      1338 non-null   object
6   charges     1338 non-null   float64
dtypes: float64(2), int64(2), object(3)
memory usage: 73.3+ KB
```

Inspecting the summary statistics

In [6]: `df.describe()`

Out[6]:

	age	bmi	children	charges
count	1338.000000	1338.000000	1338.000000	1338.000000
mean	39.207025	30.663397	1.094918	13270.422265
std	14.049960	6.098187	1.205493	12110.011237
min	18.000000	15.960000	0.000000	1121.873900
25%	27.000000	26.296250	0.000000	4740.287150
50%	39.000000	30.400000	1.000000	9382.033000
75%	51.000000	34.693750	2.000000	16639.912515
max	64.000000	53.130000	5.000000	63770.428010

Viewing the last few rows in the dataset

In [7]: `df.tail()`

Out[7]:

	age	sex	bmi	children	smoker	region	charges
1333	50	male	30.97	3	no	northwest	10600.5483
1334	18	female	31.92	0	no	northeast	2205.9808
1335	18	female	36.85	0	no	southeast	1629.8335
1336	21	female	25.80	0	no	southwest	2007.9450
1337	61	female	29.07	0	yes	northwest	29141.3603

Viewing a random sample of rows in the dataset

```
In [8]: df.sample(5)
```

```
Out[8]:
```

	age	sex	bmi	children	smoker	region	charges
115	60	male	28.595	0	no	northeast	30259.99556
847	23	male	50.380	1	no	southeast	2438.05520
44	38	male	37.050	1	no	northeast	6079.67150
129	38	male	34.700	2	no	southwest	6082.40500
464	19	male	25.175	0	no	northwest	1632.03625

Checking for missing data

```
In [9]: df.isnull().sum()
```

```
Out[9]: age          0
sex            0
bmi            0
children       0
smoker         0
region         0
charges        0
dtype: int64
```

Checking for duplicates

```
In [10]: df.duplicated().sum()
```

```
Out[10]: np.int64(1)
```

Observing the frequency of each categorical data

```
In [11]: # Observation for sex category
df.sex.value_counts()
```

```
Out[11]: sex
male      676
female    662
Name: count, dtype: int64
```

```
In [12]: # Observation for smoker category
df.smoker.value_counts()
```

```
Out[12]: smoker
no        1064
yes        274
Name: count, dtype: int64
```

```
In [13]: # Observation for region category
df.region.value_counts()
```

```
Out[13]: region
southeast    364
southwest    325
northwest    325
northeast    324
Name: count, dtype: int64
```

Investigating the data

```
In [14]: # Viewing the duplicated copy
df[df.duplicated()]
```

```
Out[14]:
```

	age	sex	bmi	children	smoker	region	charges
581	19	male	30.59	0	no	northwest	1639.5631

```
In [15]: # Comparing both duplicated records
df[df.duplicated(keep=False)]
```

```
Out[15]:
```

	age	sex	bmi	children	smoker	region	charges
195	19	male	30.59	0	no	northwest	1639.5631
581	19	male	30.59	0	no	northwest	1639.5631

Cleaning the dataset

Removing duplicated copy

```
In [16]: df.drop_duplicates(inplace=True)
```

Verifying the cleaned dataset

```
In [17]: # Checking for duplicates in the cleaned dataset
df.duplicated().sum()
```

```
Out[17]: np.int64(0)
```

```
In [18]: # Viewing the total number of rows and columns in the cleaned dataset
df.shape
```

```
Out[18]: (1337, 7)
```

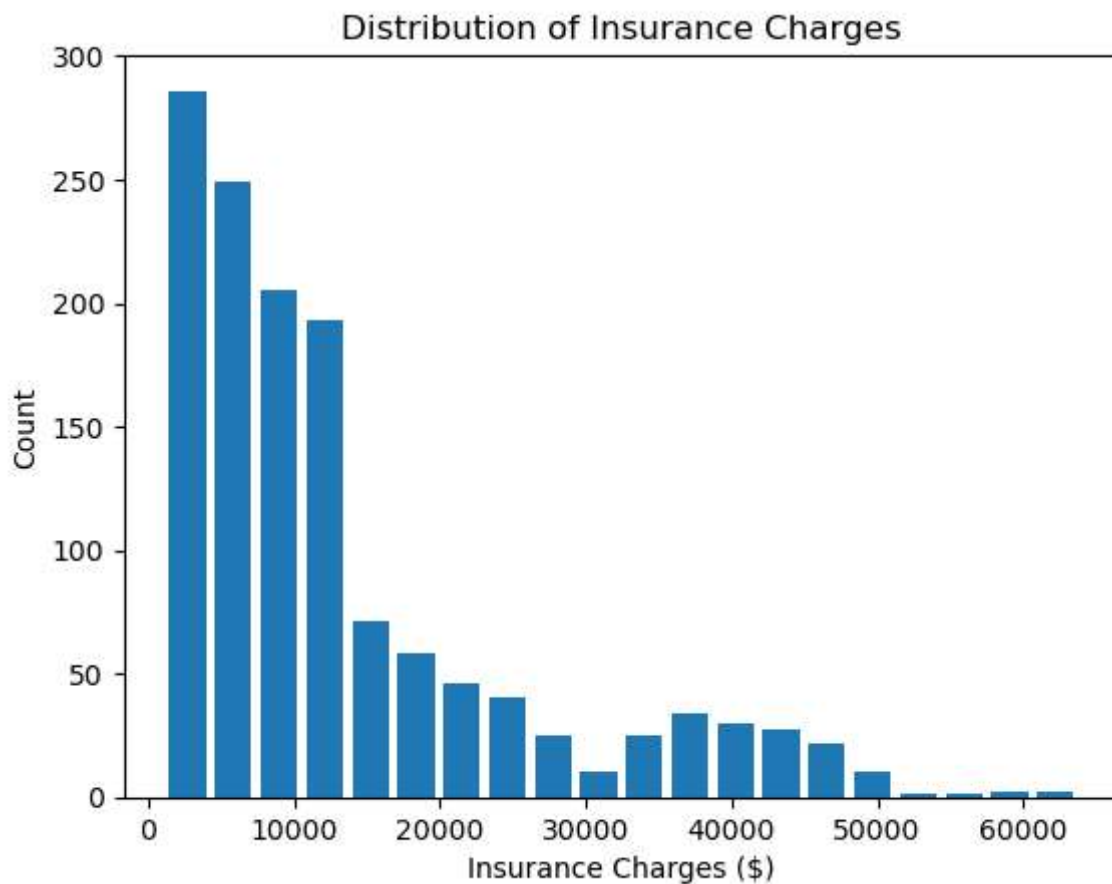
Performing exploratory data analysis (EDA)

Importing the matplotlib library

```
In [19]: import matplotlib.pyplot as plt
```

Visualizing the distribution in insurance charges

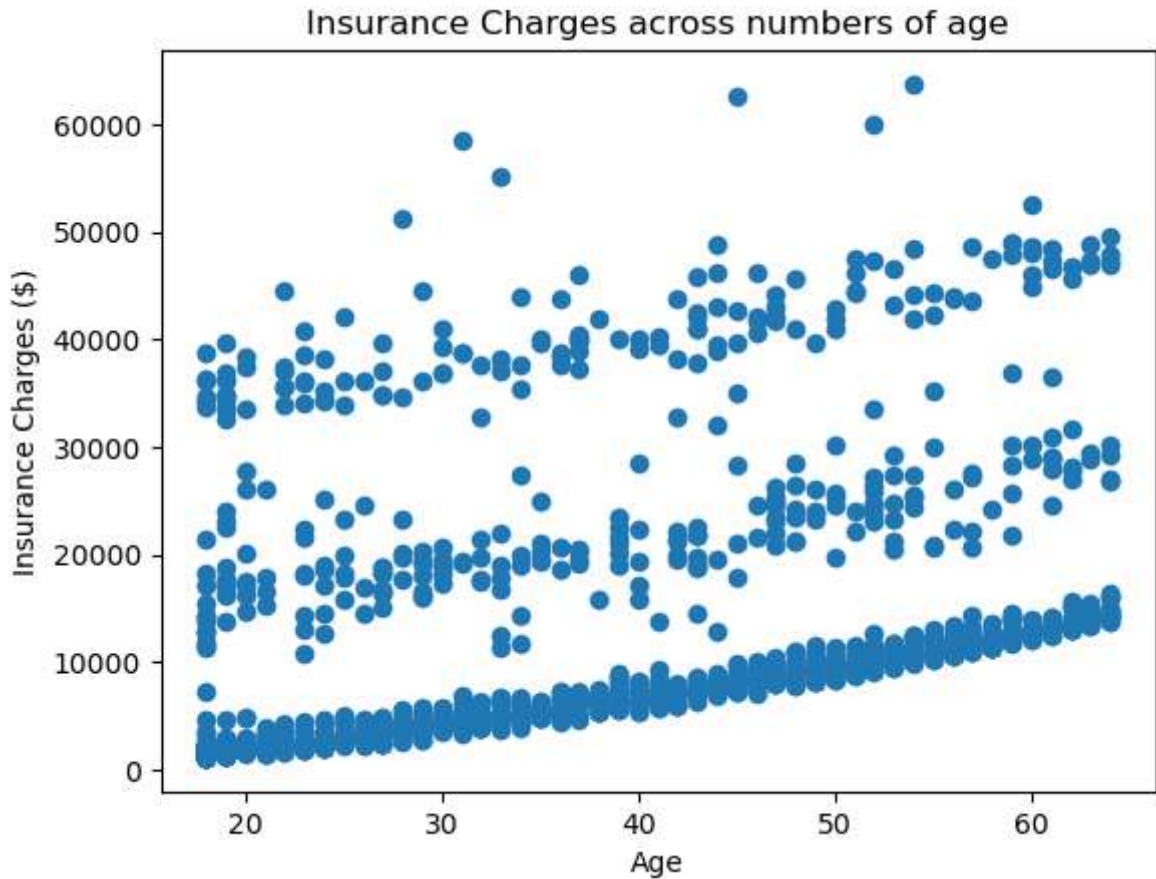
```
In [20]: # Plotting a histogram
plt.hist(df.charges, bins=20, rwidth=0.8)
plt.title('Distribution of Insurance Charges')
plt.xlabel('Insurance Charges ($)')
plt.ylabel('Count')
plt.show()
```



Visualizing the relationship between age and insurance charges

```
In [21]: # Plotting a scatter plot
plt.scatter(df.age, df.charges)
plt.title('Insurance Charges across numbers of age')
plt.xlabel('Age')
```

```
plt.ylabel('Insurance Charges ($)')  
plt.show()
```

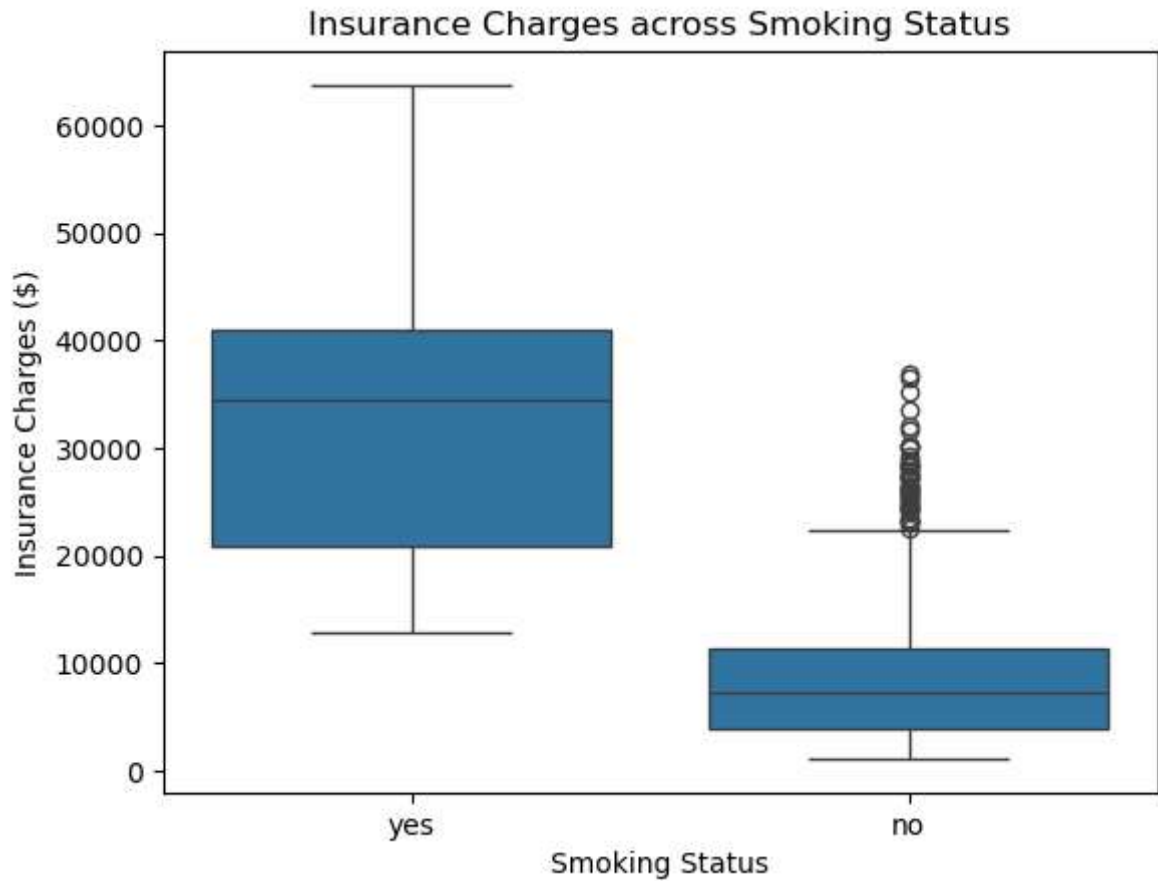


Importing the seaborn library

```
In [22]: import seaborn as sns
```

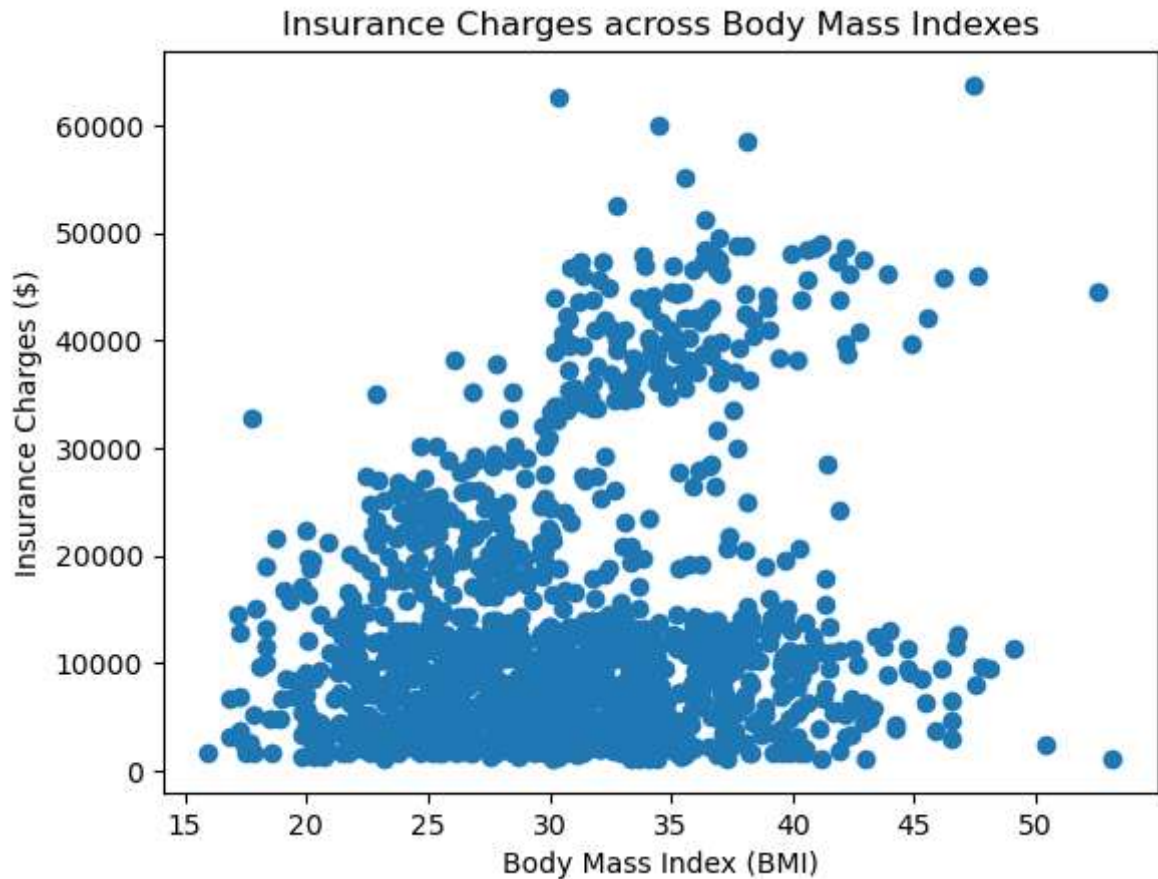
Visualizing how insurance charges differ across smoking status

```
In [23]: # Plotting a box plot  
sns.boxplot(x=df.smoker, y=df.charges)  
plt.title('Insurance Charges across Smoking Status')  
plt.xlabel('Smoking Status')  
plt.ylabel('Insurance Charges ($)')  
plt.show()
```



Visualizing the relationship between bmi and insurance charges

```
In [24]: # Plotting a scatter plot
plt.scatter(df.bmi, df.charges)
plt.title('Insurance Charges across Body Mass Indexes')
plt.xlabel('Body Mass Index (BMI)')
plt.ylabel('Insurance Charges ($)')
plt.show()
```



Finding each number of children by average insurance charges

```
In [25]: children_charge_average = df.groupby('children')['charges'].mean()
children_charge_average
```

```
Out[25]: children
0    12384.695344
1    12731.171832
2    15073.563734
3    15355.318367
4    13850.656311
5     8786.035247
Name: charges, dtype: float64
```

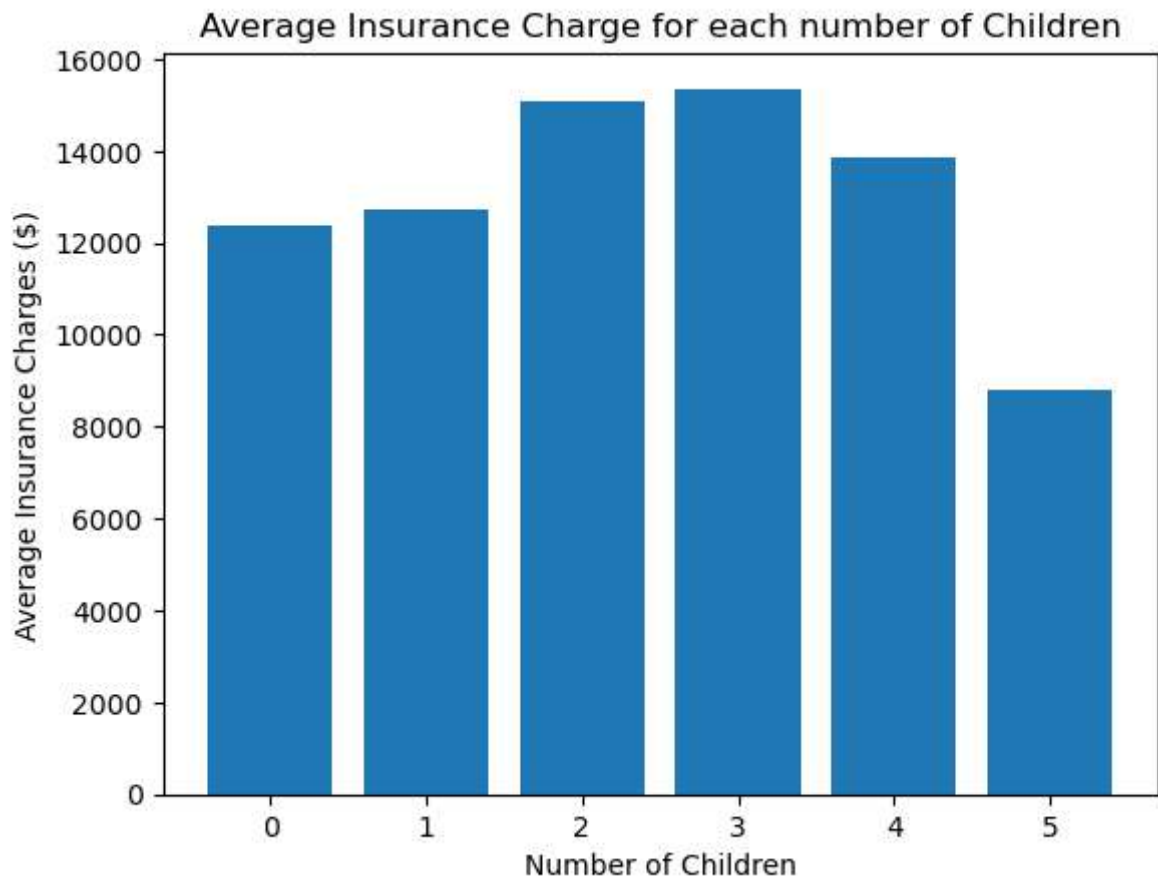
Observing the frequency of each number of children

```
In [26]: df.children.value_counts()
```

```
Out[26]: children
0     573
1     324
2     240
3     157
4       25
5       18
Name: count, dtype: int64
```

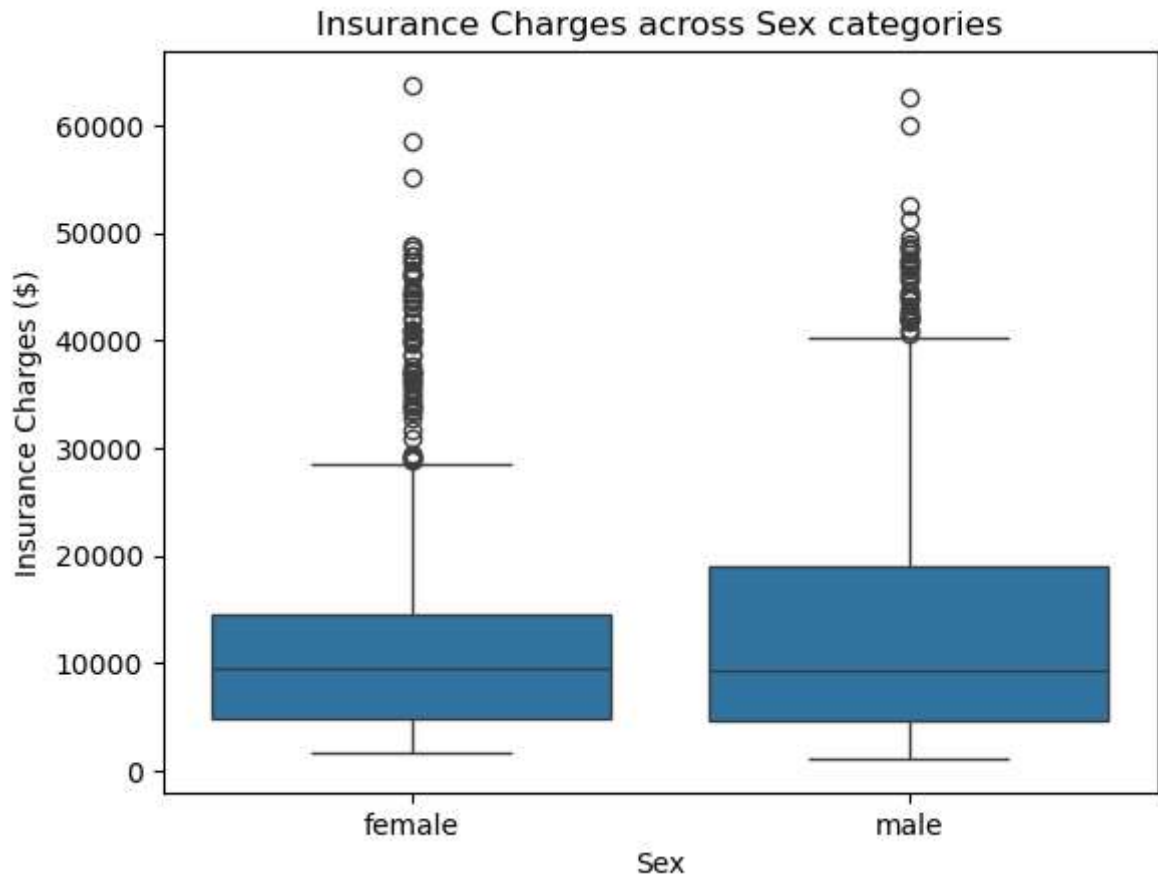
Visualizing the average insurance charge for each number of children

```
In [27]: # Plotting a bar plot
plt.bar(children_charge_average.index, children_charge_average.values)
plt.title('Average Insurance Charge for each number of Children')
plt.xlabel('Number of Children')
plt.ylabel('Average Insurance Charges ($)')
plt.show()
```



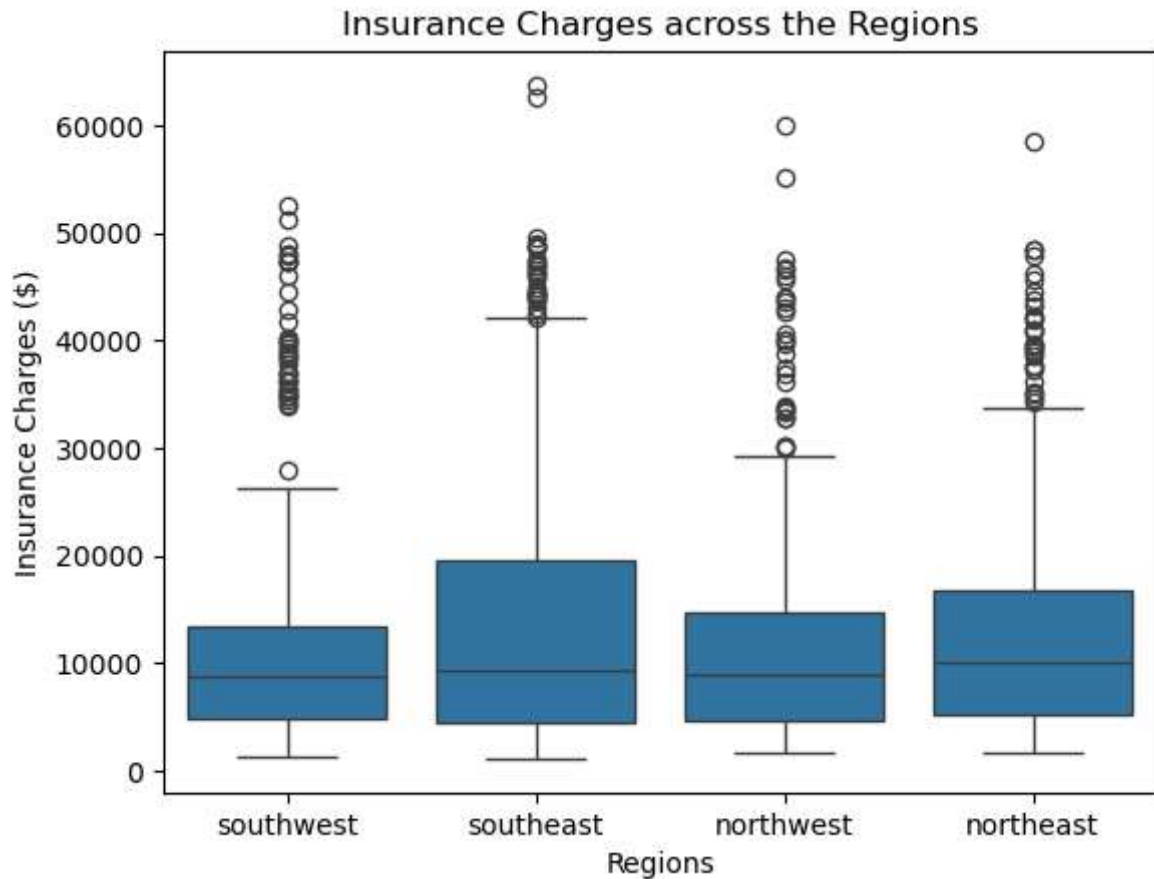
Visualizing how insurance charges differ across sex categories

```
In [28]: # Plotting a box plot
sns.boxplot(x=df.sex, y=df.charges)
plt.title('Insurance Charges across Sex categories')
plt.xlabel('Sex')
plt.ylabel('Insurance Charges ($)')
plt.show()
```



Visualizing how insurance charges differ across the regions

```
In [29]: # Plotting a box plot
sns.boxplot(x=df.region, y=df.charges)
plt.title('Insurance Charges across the Regions')
plt.xlabel('Regions')
plt.ylabel('Insurance Charges ($)')
plt.show()
```



Investigating potential outliers

```
In [30]: # Flagging observations for investigation of insurance charges using interquartile
q1 = df.charges.quantile(0.25)
q3 = df.charges.quantile(0.75)

iqr = q3 - q1

lower_boundary = q1 - (1.5 * iqr)
upper_boundary = q3 + (1.5 * iqr)

# Printing the results of the insurance charges' lower and upper boundaries
print(f'Lower boundary: {lower_boundary}, Upper boundary: {upper_boundary}')
```

Lower boundary: -13120.716174999998, Upper boundary: 34524.777625

```
In [31]: # Filtering the data to display every row whose insurance charges fall outside our
charge_outliers = df[(df['charges'] < lower_boundary) | (df['charges'] > upper_boun
charge_outliers
```

Out[31]:

	age	sex	bmi	children	smoker	region	charges
14	27	male	42.130	0	yes	southeast	39611.75770
19	30	male	35.300	0	yes	southwest	36837.46700
23	34	female	31.920	1	yes	northeast	37701.87680
29	31	male	36.300	2	yes	southwest	38711.00000
30	22	male	35.600	0	yes	southwest	35585.57600
...	...	...	...	...	...	...	...
1300	45	male	30.360	0	yes	southeast	62592.87309
1301	62	male	30.875	3	yes	northwest	46718.16325
1303	43	male	27.800	0	yes	southwest	37829.72420
1313	19	female	34.700	2	yes	southwest	36397.57600
1323	42	female	40.370	2	yes	southeast	43896.37630

139 rows × 7 columns

```
In [32]: # Observing the frequency of each group in the smoker category of potential charge
charge_outliers.smoker.value_counts()
```

```
Out[32]: smoker
yes      136
no         3
Name: count, dtype: int64
```

```
In [33]: # Investigating high-charge smokers against other smokers
smokers = df[df['smoker'] == 'yes']
smokers.shape
```

Out[33]: (274, 7)

```
In [34]: # Continuing investigation using interquartile range (IQR)
smokers_q1 = smokers['charges'].quantile(0.25)
smokers_q3 = smokers['charges'].quantile(0.75)

smokers_iqr = smokers_q3 - smokers_q1

smokers_lower_boundary = smokers_q1 - (1.5 * smokers_iqr)
smokers_upper_boundary = smokers_q3 + (1.5 * smokers_iqr)

# Printing the results of the smokers' lower and upper boundaries
print(f'Smokers lower boundary: {smokers_lower_boundary}, Smokers upper boundary: {
```

```
Smokers lower boundary: -9463.200381249997, Smokers upper boundary: 71308.6518687499
9
```

```
In [35]: # Investigating high-charge non-smokers against other non-smokers
non_smokers = df[df['smoker'] == 'no']
```

```
non_smokers.shape
```

Out[35]: (1063, 7)

```
In [36]: # Continuing investigation using interquartile range (IQR)
non_smokers_q1 = non_smokers['charges'].quantile(0.25)
non_smokers_q3 = non_smokers['charges'].quantile(0.75)

non_smokers_iqr = non_smokers_q3 - non_smokers_q1

non_smokers_lower_boundary = non_smokers_q1 - (1.5 * non_smokers_iqr)
non_smokers_upper_boundary = non_smokers_q3 + (1.5 * non_smokers_iqr)

# Printing the results of the non-smokers' lower and upper boundaries
print(f'Non-smokers lower boundary: {non_smokers_lower_boundary}, Non-smokers upper
```

Non-smokers lower boundary: -7072.319899999999, Non-smokers upper boundary: 22424.222499999996

```
In [37]: # Filtering the data to display every non-smoker observation whose charges fall out
non_smoker_outliers = non_smokers[(non_smokers['charges'] < non_smokers_lower_bound
non_smoker_outliers
```

Out[37]:

	age	sex	bmi	children	smoker	region	charges
9	60	female	25.840	0	no	northwest	28923.13692
62	64	male	24.700	1	no	northwest	30166.61817
115	60	male	28.595	0	no	northeast	30259.99556
138	54	female	31.900	3	no	southeast	27322.73386
140	34	male	22.420	2	no	northeast	27375.90478
219	24	female	23.210	0	no	southeast	25081.76784
227	58	female	41.910	0	no	southeast	24227.33724
242	55	female	26.800	1	no	southwest	35160.13457
245	54	male	30.020	0	no	northwest	24476.47851
289	52	male	26.400	3	no	southeast	25992.82104
321	26	female	29.640	4	no	northeast	24671.66334
355	46	male	27.600	0	no	southwest	24603.04837
379	62	male	31.460	1	no	southeast	27000.98473
387	50	male	25.365	2	no	northwest	30284.64294
430	19	male	33.100	0	no	southwest	23082.95533
443	59	female	36.520	1	no	southeast	28287.89766
468	28	female	24.320	1	no	northeast	23288.92840
491	61	female	25.080	0	no	southeast	24513.09126
516	20	male	35.310	1	no	southeast	27724.28875
520	50	female	27.360	0	no	northeast	25656.57526
526	19	female	30.590	2	no	northwest	24059.68019
539	53	male	31.350	0	no	southeast	27346.04207
573	62	female	36.860	1	no	northeast	31620.00106
599	52	female	37.525	2	no	northwest	33471.97189
637	35	female	38.095	2	no	northeast	24915.04626
658	48	female	35.910	1	no	northeast	26392.26029
688	47	female	24.100	1	no	southwest	26236.57997
696	53	female	32.300	2	no	northeast	29186.48236
770	61	male	36.100	3	no	southwest	27941.28758
806	40	female	41.420	1	no	northwest	28476.73499

	age	sex	bmi	children	smoker	region	charges
876	49	female	27.100	1	no	southwest	26140.36030
925	50	male	32.110	2	no	northeast	25333.33284
936	44	male	29.735	2	no	northeast	32108.66282
959	48	male	36.670	1	no	northwest	28468.91901
964	52	male	36.765	2	no	northwest	26467.09737
980	54	male	25.460	1	no	northeast	25517.11363
987	45	female	27.645	1	no	northwest	28340.18885
1008	25	male	24.985	2	no	northeast	23241.47453
1012	61	female	33.330	4	no	southeast	36580.28216
1019	21	female	32.680	2	no	northwest	26018.95052
1039	19	male	27.265	2	no	northwest	22493.65964
1098	52	female	30.875	0	no	northeast	23045.56616
1142	52	female	24.860	0	no	southeast	27117.99378
1206	59	female	34.800	2	no	southwest	36910.60803
1211	39	male	34.100	2	no	southeast	23563.01618
1258	55	male	37.715	3	no	northwest	30063.58055

```
In [38]: # Verifying the number of rows in non_smoker_outliers
non_smoker_outliers.shape[0]
```

Out[38]: 46

Preprocessing the data

Separating the dataset into independent variable (X) and dependent variable (y)

```
In [39]: # Creating our independent variable (X)
X = df.drop('charges', axis='columns')
X
```

Out[39]:

	age	sex	bmi	children	smoker	region
0	19	female	27.900	0	yes	southwest
1	18	male	33.770	1	no	southeast
2	28	male	33.000	3	no	southeast
3	33	male	22.705	0	no	northwest
4	32	male	28.880	0	no	northwest
...	...	...	...	...	...	...
1333	50	male	30.970	3	no	northwest
1334	18	female	31.920	0	no	northeast
1335	18	female	36.850	0	no	southeast
1336	21	female	25.800	0	no	southwest
1337	61	female	29.070	0	yes	northwest

1337 rows × 6 columns

```
In [40]: # Creating our dependent variable (y)
y = df.charges
y
```

```
Out[40]: 0      16884.92400
1       1725.55230
2       4449.46200
3       21984.47061
4        3866.85520
...
1333    10600.54830
1334     2205.98080
1335     1629.83350
1336     2007.94500
1337     29141.36030
Name: charges, Length: 1337, dtype: float64
```

Splitting the data into training and testing sets

```
In [41]: from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta

In [42]: # Checking the number of rows and columns in X_train, X_test, y_train, and y_test r
print(f'X_train: {X_train.shape}, X_test: {X_test.shape}, y_train: {y_train.shape},

X_train: (1069, 6), X_test: (268, 6), y_train: (1069,), and y_test: (268,)
```

Encoding Categorical variables

```
In [43]: # Replacing categorical sex data with binary codes for X_train
X_train['sex'] = X_train['sex'].map({'female': 0, 'male': 1})

# Verifying affected data
X_train['sex']
```

```
Out[43]: 1114    1
          968    1
          599    0
          170    1
          275    0
          ..
         1096    0
         1131    1
         1295    1
          861    0
         1127    0
          Name: sex, Length: 1069, dtype: int64
```

```
In [44]: # Replacing categorical smoker data with binary codes for X_train
X_train['smoker'] = X_train['smoker'].map({'no': 0, 'yes': 1})

# Verifying affected data
X_train['smoker']
```

```
Out[44]: 1114    0
          968    0
          599    0
          170    0
          275    0
          ..
         1096    1
         1131    0
         1295    0
          861    0
         1127    0
          Name: smoker, Length: 1069, dtype: int64
```

```
In [45]: # Replacing categorical sex data with binary codes for X_test
X_test['sex'] = X_test['sex'].map({'female': 0, 'male': 1})

# Verifying affected data
X_test['sex']
```

```
Out[45]: 900      1
          1064     0
          1256     0
          298      1
          237      1
          ..
          534      1
          542      0
          760      0
          1284     1
          1285     0
          Name: sex, Length: 268, dtype: int64
```

```
In [46]: # Replacing categorical smoker data with binary codes for X_test
X_test['smoker'] = X_test['smoker'].map({'no': 0, 'yes': 1})

# Verifying affected data
X_test['smoker']
```

```
Out[46]: 900      0
          1064     0
          1256     0
          298      1
          237      0
          ..
          534      0
          542      0
          760      0
          1284     1
          1285     0
          Name: smoker, Length: 268, dtype: int64
```

```
In [47]: # Performing One-Hot Encoding on categorical region data
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer

ct = ColumnTransformer([('region', OneHotEncoder(drop='first'), ['region'])], remain
```

```
In [48]: # Training ColumnTransformer on X_train
X_train = ct.fit_transform(X_train)

# Verifying the newly transformed data
X_train
```

```
Out[48]: array([[ 0.    ,  0.    ,  0.    , ..., 24.51 ,  0.    ,  0.    ],
 [ 0.    ,  0.    ,  0.    , ..., 25.745,  2.    ,  0.    ],
 [ 1.    ,  0.    ,  0.    , ..., 37.525,  2.    ,  0.    ],
 ...,
 [ 0.    ,  0.    ,  1.    , ..., 22.    ,  1.    ,  0.    ],
 [ 0.    ,  0.    ,  1.    , ..., 28.    ,  3.    ,  0.    ],
 [ 0.    ,  1.    ,  0.    , ..., 35.86 ,  2.    ,  0.    ]],
          shape=(1069, 8))
```

```
In [49]: # Applying the trained ColumnTransformer on X_test
X_test = ct.transform(X_test)
```

```
# Verifying the newly transformed data
X_test
```

```
Out[49]: array([[ 0.    ,  0.    ,  0.    , ..., 22.515,  0.    ,  0.    ],
 [ 0.    ,  0.    ,  1.    , ..., 25.6   ,  4.    ,  0.    ],
 [ 1.    ,  0.    ,  0.    , ..., 36.385,  3.    ,  0.    ],
 ...,
 [ 0.    ,  0.    ,  0.    , ..., 34.58  ,  2.    ,  0.    ],
 [ 0.    ,  0.    ,  1.    , ..., 36.3   ,  1.    ,  1.    ],
 [ 0.    ,  0.    ,  0.    , ..., 24.32  ,  0.    ,  0.    ]],
      shape=(268, 8))
```

Building our model

```
In [50]: # Importing LinearRegression
from sklearn.linear_model import LinearRegression

# Creating an instance of the model
lr = LinearRegression()
```

Training our model

```
In [51]: lr.fit(X_train, y_train)
```

```
Out[51]:
```

▼ LinearRegression ⓘ ?

▶ Parameters

Testing our model

```
In [52]: lr.predict(X_test)
```

```

Out[52]: array([ 8.14369388e+03,  5.73711568e+03,  1.43693149e+04,  3.17455136e+04,
 8.96238666e+03,  1.31497224e+04,  3.04467607e+04,  1.45328881e+03,
 1.06330184e+04,  1.13189438e+04,  1.03778536e+04,  3.31184377e+04,
 3.10772527e+04,  1.74119253e+04,  1.08016743e+04,  9.52889964e+03,
 4.16103784e+03,  3.17315373e+04,  3.21938875e+03,  5.22992460e+03,
 3.54979004e+03,  3.02837740e+04,  1.48989509e+04,  3.04569093e+04,
 3.11077668e+04,  5.51196081e+03,  3.55030401e+04,  3.65704810e+04,
 1.14123133e+04,  1.42056990e+04,  6.50234029e+03,  1.27246945e+04,
 3.99806590e+02,  1.20883980e+04,  3.96592069e+04,  1.23395540e+04,
 4.74412279e+03,  3.91903157e+03,  3.08606789e+04,  8.95870663e+03,
 7.05389441e+03,  3.01125636e+04,  3.47804324e+04,  1.22916189e+04,
 7.35739310e+03,  3.42060530e+03,  6.21243471e+03,  8.98345200e+03,
 4.26401828e+03,  9.01992870e+03,  6.60149578e+03,  1.20802337e+04,
 3.12447687e+04,  3.55341947e+03,  1.10470477e+04,  9.73512492e+03,
 1.08383403e+04,  2.42942719e+03,  3.11621909e+04,  9.40578078e+03,
 1.55793421e+04,  8.19158958e+03,  1.26009604e+04,  1.59967376e+03,
 1.70001498e+04,  1.06976951e+04,  1.05787114e+04,  3.07573915e+04,
 2.47684641e+04,  1.65767575e+04,  7.26593462e+03,  2.97283456e+03,
 7.14516766e+03,  1.17475472e+01,  1.16917695e+04, -1.10794156e+03,
 1.71191624e+04,  8.61743629e+03,  7.73188392e+03,  1.28117166e+04,
 5.89112422e+03,  1.85397278e+04,  1.54760606e+04,  1.09593149e+04,
 3.64631656e+04,  1.31427713e+04,  4.36626594e+03,  2.32795031e+03,
 6.66786737e+03,  4.05265152e+03,  1.50107639e+04,  2.81775850e+04,
 5.50606102e+03,  3.27070584e+03,  2.17468328e+03,  3.59080511e+04,
 1.43386030e+04,  8.85353515e+03,  4.19006954e+03,  8.70129717e+03,
 1.40201891e+04,  1.51506182e+04,  3.21262598e+04,  1.45653368e+04,
 3.27490687e+04,  6.33977275e+01,  3.11302368e+04,  2.67595226e+04,
 1.39208487e+04,  6.00386014e+03,  8.13424910e+03,  1.15709262e+04,
 8.47634509e+03,  1.02630067e+04,  6.19600866e+03,  5.35257699e+03,
 1.96956852e+03,  3.05699803e+03,  7.03250999e+03,  9.04721600e+03,
 3.60160730e+04,  8.22939170e+03,  1.15509446e+04,  9.19112466e+03,
 9.02857998e+03,  9.65151180e+03,  4.63394829e+03,  1.23764425e+04,
 2.97975708e+04,  3.89782922e+04,  3.59988557e+04,  1.22783148e+04,
 2.37552913e+03,  1.14870367e+04,  1.04356777e+04,  5.49847080e+03,
 3.49280437e+04,  2.98883176e+04,  2.55134521e+04,  4.85814735e+03,
 6.75427392e+03,  5.78232721e+03,  3.65432602e+04,  1.08563332e+04,
 5.54605628e+03,  8.38183330e+03,  1.02229130e+04,  8.46677926e+03,
 3.50191403e+04,  1.44283736e+04,  9.83704544e+03,  6.82474848e+03,
 7.08744504e+03,  1.67861413e+04,  6.91930980e+03,  2.64144011e+04,
 3.67459778e+04,  1.67110391e+04,  9.90371852e+03,  1.10883601e+04,
 1.27258897e+04,  4.17075074e+03,  1.28062707e+04,  1.35691842e+04,
 3.56720988e+04,  3.33291421e+04,  1.28469818e+04,  2.51973637e+04,
 2.59094502e+03,  2.06618440e+03,  4.90710838e+03,  5.01656948e+03,
 5.25500056e+03,  9.72148167e+03,  1.00340209e+04,  2.62638450e+04,
 2.40736111e+03,  1.06048617e+04,  6.84994523e+03,  1.21285805e+04,
 1.49149746e+04,  5.09974308e+03,  1.08865915e+04,  1.00510967e+04,
 3.30332394e+04,  1.08761351e+04,  7.27836589e+03,  5.87607955e+03,
 4.07152877e+02,  3.15538451e+04,  8.28295276e+03,  3.27235841e+04,
 1.26593388e+04,  5.97455676e+03,  1.76176909e+03,  2.04034043e+03,
 3.27904488e+04,  5.02841772e+03,  1.31813516e+04,  1.75033637e+04,
 2.63807625e+04,  2.57357428e+04,  6.53956113e+03,  1.21356232e+04,
 2.84941613e+04,  7.96062099e+03,  1.02945154e+04,  1.02754138e+04,
 3.18631733e+03,  3.43106363e+04,  3.15743820e+04,  9.63573589e+03,
 5.80208158e+03,  7.64845273e+03,  1.07788176e+04,  4.73611221e+03,
 5.11202684e+03,  1.19303968e+04,  9.24148526e+03,  3.03338784e+03,
 4.22588341e+03,  3.75583797e+04,  3.71212906e+04,  1.32569289e+04,

```

```

1.07292470e+04, 3.77277415e+04, 3.44367796e+04, 6.92689399e+03,
2.70390160e+04, 4.08364156e+03, 1.57311276e+04, 2.94795994e+04,
5.77344989e+03, 9.43923528e+03, 8.79720762e+03, 9.37279417e+03,
9.64338184e+03, 4.28036915e+03, 5.71127345e+03, 4.59046392e+03,
6.29714185e+03, 4.76798318e+03, 5.98323724e+03, 8.81750085e+03,
1.15335666e+03, 8.41676111e+03, 3.79695094e+03, 1.09242855e+04,
2.67006819e+03, 1.02838290e+04, 3.90727328e+03, 1.19204157e+04,
1.22497808e+04, 1.45122628e+04, 3.69882154e+04, 2.32241598e+04,
1.01400918e+04, 7.91971044e+03, 9.18658937e+03, 1.25158649e+04,
2.94087920e+04, 3.78651596e+04, 1.47262168e+04, 1.67534065e+04,
1.52745658e+04, 6.45469935e+03, 3.84671567e+04, 8.32407060e+03])

```

Evaluating our model performance

```
In [53]: lr.score(X_test, y_test)
```

```
Out[53]: 0.8069287081198014
```

Inspecting coefficients our model learned

```
In [54]: lr.coef_
```

```
Out[54]: array([ -391.76145478, -838.91961573, -659.13975155,  248.21072022,
        -101.54205399,   318.70144095,   533.0099888 , 23077.76459287])
```

```
In [55]: # Getting the exact feature order corresponding to the eight coefficients
ct.get_feature_names_out()
```

```
Out[55]: array(['region__region_northwest', 'region__region_southeast',
        'region__region_southwest', 'remainder__age', 'remainder__sex',
        'remainder__bmi', 'remainder__children', 'remainder__smoker'],
        dtype=object)
```

Inspecting the intercept our model learned

```
In [56]: lr.intercept_
```

```
Out[56]: np.float64(-11092.652295946064)
```